



Linear, Binary, and Interpolation Search using Arrays and Linked Lists; Elementary Search Techniques

WEEK-4/5

DATE: 8-12/09/2025

Linear Search Using LL

- ▶ Already Discussed in previous Lecture.

Binary Search Using LL

- ▶ Binary Search **requires random access** to the middle element (like arrays do), but **Linked Lists don't support random access** — we must traverse sequentially.
- ▶ So, Binary Search on a Linked List is possible, but it's **not efficient** (it becomes $O(n \log n)$ instead of $O(\log n)$).

Binary Search Using LL(Pseudo Code)

FUNCTION BinarySearchLinkedList(head, key):

start ← head

end ← NULL

WHILE (start ≠ end):

mid ← GetMiddle(start, end)

IF mid = NULL:

RETURN NULL // element not found

IF mid.data = key:

RETURN mid // element found

ELSE IF mid.data < key:

start ← mid.next // search right half

ELSE:

end ← mid // search left half

RETURN NULL // element not found

FUNCTION GetMiddle(start, end):

IF start = NULL:

RETURN NULL

slow ← start

fast ← start.next

WHILE (fast ≠ end):

fast ← fast.next

IF (fast ≠ end):

slow ← slow.next

fast ← fast.next

RETURN slow

Binary Search Using LL(Pseudo Code)

- In binary search does we need sorted list?

Yes — just like with binary search in arrays, the linked list must be sorted first. Binary search only works correctly on sorted data, otherwise the "divide and search" logic fails.

EXAMPLE:

Sorted Linked List:

10 → 20 → 30 → 40 → 50 → NULL

Example:

Iteration 1:

start = 10,

end = NULL

Middle = 30 (found using slow/fast pointer)

Compare: $\text{mid.data} = 30 == \text{key}(30)$ Found $\rightarrow$ return node

Iteration 2:

Example if searching for 40:

First middle = 30

$30 < 40 \rightarrow$ search right side $\rightarrow$ start = 40

Next middle = 40

$40 == 40 \rightarrow$ Found

Iteration 3:

Example if searching for 15:

First middle = 30

$15 < 30 \rightarrow$ search left side (end = 30)

Middle between 10 and 20 = 20

Compare: $20 > 15 \rightarrow$ search left side (end = 20)

Middle between 10 and 10 = 10

Compare: $10 < 15 \rightarrow$ search right side $\rightarrow$ start = 20

Now start == end $\rightarrow$ Stop $\rightarrow$ Not found.

Code Provided on GCR

Linear & Binary Search in Arrays

- ▶ Discussed in lab in detail according to arrays.

Interpolation Search Using Arrays

- ▶ What is Interpolation Search?
 - ▶ A searching algorithm (like Binary Search).
 - ▶ Works only on sorted arrays (ascending order).
 - ▶ Instead of always checking the middle index, it estimates the probable position of the key using the values.
 - ▶ Best when data is uniformly distributed.
- ▶ **Formula:**
$$\text{pos} = \text{low} + ((\text{key} - \text{arr}[\text{low}]) * (\text{high} - \text{low})) / (\text{arr}[\text{high}] - \text{arr}[\text{low}])$$
 - ▶ low = starting index
 - ▶ high = ending index
 - ▶ arr[low] = value at low index
 - ▶ arr[high] = value at high index
 - ▶ key = value we are searching for

Interpolation Search Using Arrays

- ▶ What is Interpolation Search?
 - ▶ A searching algorithm (like Binary Search).
 - ▶ Works only on sorted arrays (ascending order).
 - ▶ Instead of always checking the middle index, it estimates the probable position of the key using the values.
 - ▶ Best when data is uniformly distributed.
- ▶ **Formula:**
$$\text{pos} = \text{low} + ((\text{key} - \text{arr}[\text{low}]) * (\text{high} - \text{low})) / (\text{arr}[\text{high}] - \text{arr}[\text{low}])$$
 - ▶ low = starting index
 - ▶ high = ending index
 - ▶ arr[low] = value at low index
 - ▶ arr[high] = value at high index
 - ▶ key = value we are searching for

Pseudo Code

FUNCTION InterpolationSearch(arr, n, key):

low $\leftarrow$ 0

high $\leftarrow$ n - 1

WHILE low $\leq$ high AND key $\geq$ arr[low] AND key $\leq$ arr[high]:

pos $\leftarrow$ low + ((key - arr[low]) * (high - low)) / (arr[high] - arr[low])

IF arr[pos] == key:

RETURN pos // found

ELSE IF arr[pos] < key:

low $\leftarrow$ pos + 1

ELSE:

high $\leftarrow$ pos - 1

RETURN -1 // not found

Code Provided on GCR

Interpolation Search Using LL

- ▶ Interpolation Search is like Binary Search, but instead of always checking the middle, it estimates the position using a formula (like "guessing" where the key should be).
- ▶ In an array, it uses indices, this works because arrays give random access (we can jump to `arr[pos]` directly).

Problem with Linked List

- ▶ In a linked list, we cannot directly jump to the calculated pos.
- ▶ We would have to traverse from the start each time $\rightarrow O(n)$ just to reach the position.
- ▶ This makes Interpolation Search inefficient on LL (even worse than Binary Search).

Pseudo Code

FUNCTION InterpolationSearchLinkedList(head, key):

low $\leftarrow$ head

high $\leftarrow$ lastNode(head)

WHILE (low $\neq$ NULL AND high $\neq$ NULL AND key $\geq$ low.data AND key $\leq$ high.data):

// Calculate position (using values, not indices)

posIndex = ((key - low.data) * (index(high) - index(low)))
/ (high.data - low.data)

posNode = moveFrom(low, posIndex) // traverse from low to posIndex

IF posNode.data == key:

RETURN posNode

ELSE IF posNode.data < key:

low $\leftarrow$ posNode.next

ELSE:

high $\leftarrow$ posNode.prev (if DLL) or recompute high

RETURN NULL // Not found

Algorithm Steps

- Set low = 0, high = n-1.
- While low $\leq$ high and key lies between arr[low] and arr[high]:
- Estimate position pos using formula.
- If arr[pos] == key $\rightarrow$ Found.
- If arr[pos] < key $\rightarrow$ search right half (low = pos + 1).
- If arr[pos] > key $\rightarrow$ search left half (high = pos - 1).
- If loop ends $\rightarrow$ Not found.

Interpolation Search Using LL

► Why It's Inefficient

- In **arrays** → $O(\log \log n)$ (very fast when data is uniformly distributed).
 - In **linked lists** → Each probe needs traversal $O(n)$.
 - Overall complexity becomes **$O(n \log \log n)$** , which is **slower** than simple Binary Search on LL.
- Interpolation Search is not practical on Linked Lists because they don't allow random access.
- Better to stick with linear search for unsorted LL, or binary search for sorted LL (even though it's $O(n \log n)$).

Code Provided on GCR

Time Complexities

Search Type	Array	Linked List	Time
Linear Search	✓	✓ (best choice)	$O(n)$
Binary Search	✓	✓ (but inefficient)	$O(n \log n)$
Interpolation Search	✓ (only sorted, uniform data)	✗ Not possible	$O(n \log \log n)$,

Types of Time Complexities

1. Constant Time – $O(1)$ Definition:

- ▶ Execution time does not depend on the input size n .
- ▶ Example: Accessing an element in an array by index, `arr[5]`.
- ▶ Meaning: Always takes the same amount of time.

2. Logarithmic Time – $O(\log n)$

- ▶ **Definition:** Execution time grows logarithmically with input size.
- ▶ **Example:** Binary search on a sorted array.
- ▶ **Meaning:** Each step cuts the problem size in half.

3. Linear Time – $O(n)$

- ▶ **Definition:** Execution time grows proportionally with input size.
- ▶ **Example:** Traversing a linked list or array.
- ▶ **Meaning:** If input doubles, runtime doubles.

Types of Time Complexities

4. Linearithmic Time – $O(n \log n)$

- ▶ Definition: Execution time grows like n times $\log n$.
- ▶ Example: Merge Sort, Quick Sort (average case), Heap Sort.
- ▶ Meaning: Faster than quadratic but slower than linear.

5. Quadratic Time – $O(n^2)$

- ▶ **Definition:** Execution time grows proportional to the square of input size.
- ▶ **Example:** Bubble Sort, Insertion Sort, Selection Sort (worst case).
- ▶ **Meaning:** Nested loops over the same input size.

6. Cubic Time – $O(n^3)$

- ▶ **Definition:** Execution time grows proportional to the cube of input size.
- ▶ **Example:** Certain dynamic programming problems, matrix multiplication (naïve).
- ▶ **Meaning:** Triple nested loops.

Types of Time Complexities

7. Polynomial Time – $O(n^k)$ Definition:

- ▶ Execution time grows as a polynomial function of input size.
- ▶ Example: n^4 , n^5 etc. (rare in practice).
- ▶ Meaning: Still considered feasible but expensive.

8. Exponential Time – $O(2^n)$ Definition:

- ▶ Execution time doubles with each additional input element.
- ▶ Example: Recursive Fibonacci, brute force subset generation.
- ▶ Meaning: Extremely slow, impractical for large n .

9. Factorial Time – $O(n!)$

- ▶ **Definition:** Execution time grows as factorial of input size.
- ▶ **Example:** Solving Traveling Salesman Problem (TSP) using brute force, generating all permutations.
- ▶ **Meaning:** The slowest common complexity, grows faster than exponential.

Types of Time Complexities

Complexity	Growth Rate	Example
$O(1)$	Constant	Access array element
$O(\log n)$	Logarithmic	Binary search
$O(n)$	Linear	Traversing array
$O(n \log n)$	Linearithmic	Merge Sort
$O(n^2)$	Quadratic	Bubble Sort
$O(n^3)$	Cubic	Matrix multiplication
$O(n^k)$	Polynomial	General polynomial loops
$O(2^n)$	Exponential	Recursive Fibonacci
$O(n!)$	Factorial	Traveling Salesman brute force



Elementary Sorting Techniques: (Bubble Sort, Selection Sort, Insertion Sort, Comb Sort)

Bubble Sort

- ▶ Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order.
- ▶ After each pass, the largest element "bubbles up" to its correct position at the end.
- ▶ Worst-case: $O(n^2)$, Best: $O(n)$, Average: $O(n^2)$.
- ▶ Pseudo Code:

```
procedure bubbleSort(arr, n)
  repeat
    swapped = false
    for i = 0 to n-2
      if arr[i] > arr[i+1]
        swap(arr[i], arr[i+1])
        swapped = true
    end //for
  until swapped == false
end //procedure
```

C++ Code:

```
void bubbleSort(int arr[], int n) {
    for(int i = 0; i < n-1; i++) {
        for(int j = 0; j < n-i-1; j++) {
            if(arr[j] > arr[j+1]) {
                swap(arr[j], arr[j+1]);
            }
        }
    }
}
```

Selection Sort

- ▶ Selection Sort selects the minimum element from the unsorted part and swaps it with the first unsorted element.
- ▶ After each pass, one element is placed in its correct position.
- ▶ Worst-case: $O(n^2)$. Best: $O(n^2)$, Average: $O(n^2)$.
- ▶ Pseudo Code:

```
procedure selectionSort(arr, n)  
  for i = 0 to n-2  
    minIndex = i  
    for j = i+1 to n-1  
      if arr[j] < arr[minIndex]  
        minIndex = j  
    end for  
    swap(arr[i], arr[minIndex])  
  end for  
end procedure
```

C++ Code:

```
void selectionSort(int arr[], int n) {  
    for(int i = 0; i < n-1; i++) {  
        int minIndex = i;  
        for(int j = i+1; j < n; j++) {  
            if(arr[j] < arr[minIndex])  
                minIndex = j;  
        }  
        swap(arr[i], arr[minIndex]);  
    }  
}
```

Insertion Sort

- ▶ In each iteration, it **takes the next element** from the array (the element at index i).
- ▶ That element becomes the **key**.
- ▶ Then, it is **compared backwards** with the already sorted part (from index 0 to $i-1$) and inserted into the correct position.
- ▶ So the key is always **the current element in order (from left to right)** — never random.
- ▶ Worst-case: $O(n^2)$. Best: $O(n)$, Average: $O(n^2)$.
- ▶ **Key point:** Insertion Sort **always selects the next element in sequence** as the key, then "inserts" it into the sorted sublist.

Insertion Sort

► Pseudo Code:

procedure insertionSort(arr, n)

for i = 1 **to** n-1

key = arr[i]

 j = i - 1

while j >= 0 **AND** arr[j] > key

 arr[j+1] = arr[j]

 j = j - 1

end while

 arr[j+1] = key

end for

end procedure

C++ Code:

```
void insertionSort(int arr[], int n) {  
  // Loop through array, starting from 2nd element  
  for(int i = 1; i < n; i++) {  
    int key = arr[i];      // Key = element to be inserted  
    int j = i - 1;         // Compare with elements before i  
  
    // Shift elements larger than key one step right  
    while(j >= 0 && arr[j] > key) {  
      arr[j+1] = arr[j];  
      j --;  
    }  
    arr[j+1] = key;        // Insert key into its correct position  
  }  
}
```

Comb Sort

- ▶ Comb Sort is an improvement over Bubble Sort.
- ▶ Instead of just comparing adjacent elements, it uses a gap that shrinks after each pass (typically $\text{gap} = \text{gap} / 1.3$).
- ▶ When gap becomes 1, it behaves like Bubble Sort.
- ▶ Worst-case: $O(n^2)$. Best: $O(n \log n)$, Average: $O(n^2)$.
- ▶ **Pseudo Code:**

Comb Sort

► Pseudo Code:

```
procedure combSort(arr, n)
    gap = n; shrink = 1.3; sorted = false;
    while NOT sorted
        gap = floor(gap / shrink)
        if gap < 1
            gap = 1
        end if
        sorted = (gap == 1)
        i = 0
        while (i + gap) < n
            if arr[i] > arr[i + gap]
                swap(arr[i], arr[i + gap])
                sorted = false
            end if
            i = i + 1
        end while
    end while
end procedure
```

C++ Code:

```
void combSort(int arr[], int n) {
    int gap = n;
    bool sorted = false;
    const float shrink = 1.3;

    while(!sorted) {
        gap = int(gap / shrink);
        if(gap <= 1) {
            gap = 1;
            sorted = true;
        }
        for(int i = 0; i + gap < n; i++) {
            if(arr[i] > arr[i+gap]) {
                swap(arr[i], arr[i+gap]);
                sorted = false;
            }
        }
    }
}
```

Bubble Sort in LL

- ▶ Compare **current node's data** with its **next node's data**.
- ▶ If they are in the wrong order, **swap their values**.
- ▶ Keep doing this until no swaps are needed (list is sorted).
- ▶ This “bubbles” the largest value to the end in each pass.
- ▶ Same idea as array bubble sort, but instead of indexes, we use **pointers** (current and current->next).

C++ Code:

```
void bubbleSort(int arr[], int n) {  
    for(int i = 0; i < n-1; i++) {  
        for(int j = 0; j < n-i-1; j++) {  
            if(arr[j] > arr[j+1]) {  
                swap(arr[j], arr[j+1]);  
            }  
        }  
    }  
}
```

Bubble Sort in LL

► Pseudo Code

```
procedure bubbleSortSLL(head)
  if head == NULL then return

  repeat
    swapped = false
    current = head
    while current.next != NULL
      if current.data > current.next.data
        swap(current.data, current.next.data)
        swapped = true
      end if
      current = current.next
    end while
  until swapped == false
end procedure
```

C++ Code:

```
void bubbleSort(Node* head) {
  if (!head) return;
  bool swapped;
  do {
    swapped = false;
    Node* curr = head;
    while (curr->next != NULL) {
      if (curr->data > curr->next->data) {
        // just swap values
        swap(curr->data, curr->next->data);
        swapped = true;
      }
      curr = curr->next;
    }
  } while (swapped);
}
```

Bubble Sort in LL

Comparison: Arrays vs SLL

- ▶ **Arrays:** Bubble Sort is $O(n^2)$ but comparisons are fast because of **random access**.
- ▶ **SLL:** Bubble Sort is still $O(n^2)$, but a bit slower in practice because:
 - ▶ Sequential access only.
 - ▶ Pointer dereferencing (current->next) adds overhead.
 - ▶ Cache-unfriendly (nodes are scattered in memory).

Final Complexity for Bubble Sort on SLL:

- ▶ **Best case:** $O(n)$
- ▶ **Average case:** $O(n^2)$
- ▶ **Worst case:** $O(n^2)$
- ▶ **Space complexity:** $O(1)$